Modeling and Control of a Simplified Hopper Robot

Konstantinos Spinos

***Abstract*—Quadratic Programming (QP) refers to optimization problems that comprise quadratic functions bound to linear constraints. QP is a common type of non-linear programming used to optimize such problems. The goal of a QP is to find the optimal state that minimizes the quadratic cost while satisfying all constraints. Augmented Lagrangian Methods (ALM) can be used in such cases. This paper focuses on Primal-Dual ALM with slack variables, a method that minimizes the quadratic function with respect to both the primal and the dual variables simultaneously.**

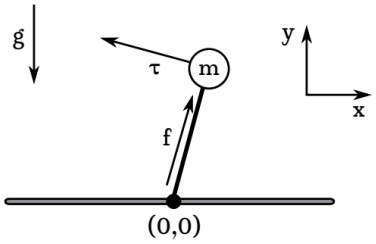
I. Introduction

In this paper, we will attempt to build the model and simulation of a simplified hopper robot. The system will be controlled firsty by a PD controller and, afterwards, we will implement the QP solver for a more robust approach.

II. Modeling

The system "lives" in the 2D plane, and has two control inputs: a force, f, applied between the body and the ground, and an another force, τ , applied to the body perpendicular to the first force. The state of the system and the control inputs are as follows:

$$x = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix} \in \mathbb{R}^4 \qquad u = \begin{bmatrix} f \\ \tau \end{bmatrix} \in \mathbb{R}^2$$



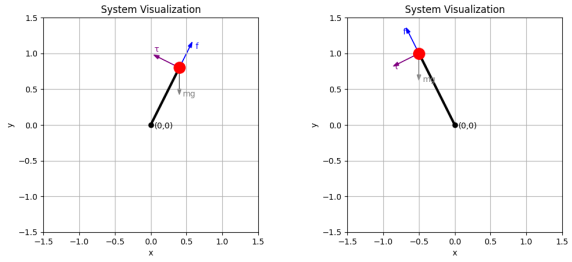
Visual Representation of the Simplified Hopper Robot.
The base of the robot (0,0) is fixed while the rod's length is dynamic.
The continuous dynamics of the system are given by the following equations:

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \ddot{x} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \frac{r_x f - r_y \tau}{m} \\ \frac{r_y f + r_x \tau}{m} - g \end{bmatrix} \in \mathbb{R}^4$$

where $g = 9.81$ is the gravity, $m = 5kg$ is the body mass and

$$\mathbf{r} = \begin{bmatrix} r_x \\ r_y \end{bmatrix} = \frac{\begin{bmatrix} x & y \end{bmatrix}^T}{\sqrt{x^2 + y^2}}$$

A simple visualization of the system can be created in python environment using the native library *matplotlib*.



The system alone is not very interesting. The next part consists of building and tuning a PD controller to derive a desired state of the system by applying the right control inputs **u**. We discretize the system using:
1) Euler Integration

$$x_{k+1} = x_k + f(x_k, u_k)dt$$

2) Semi-Implicit Euler Integration

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \ddot{x} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}}_k \\ \dot{\mathbf{v}}_k \end{bmatrix} = f(x_k, u_k)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \dot{\mathbf{v}}_k \ dt$$
$$\mathbf{q}_{k+1} = \mathbf{q}_k + \mathbf{v}_{k+1} \ dt$$

3) Runge Kutta 4th Order Integration

$$f_1 = f(x_k, u_k)$$
$$f_2 = f(x_k + f_1 \frac{dt}{2}, u_k)$$
$$f_3 = f(x_k + f_2 \frac{dt}{2}, u_k)$$
$$f_4 = f(x_k + f_3 dt, u_k)$$
$$x_{k+1} = x_k + \frac{dt}{6}(f_1 + 2f_2 + 2f_3 + f_4)$$

4) Midpoint Integration

$$x_m = x_k + f(x_k, u_k) \ \frac{dt}{2}$$
$$x_{k+1} = x_k + f(x_m, u_k)dt$$

We can now apply the PD controller to the system. For this, we shall choose a derired pose (reference point) for the mass system - let us choose, for convenience, that we only wish to control the position part of the system (e.g. velocity is set to

zero). Given the initial state $\mathbf{x} = \begin{bmatrix}0 & 1 & 0 & 0\end{bmatrix}^T$, we can plan the PD controller as seen above:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \end{bmatrix} \Rightarrow \begin{aligned} \mathbf{q} &= \begin{bmatrix} x \\ y \end{bmatrix} \quad \rightarrow \text{ Position part} \\ \mathbf{v} &= \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} \quad \rightarrow \text{ Velocity part} \end{aligned}$$

$$\mathbf{x_{des}} = \begin{bmatrix} x_d \\ y_d \\ \dot{x_d} \\ \dot{y_d} \end{bmatrix} \Rightarrow \begin{aligned} \mathbf{q_{des}} &= \begin{bmatrix} x_d \\ y_d \end{bmatrix} \quad \rightarrow \text{ Desired position} \\ \mathbf{v_{des}} &= \begin{bmatrix} \dot{x_d} \\ \dot{y_d} \end{bmatrix} \quad \rightarrow \text{ Desired velocity} \end{aligned}$$

Since we wish to control the input forces $\mathbf{u} = \begin{bmatrix} f & \tau \end{bmatrix}^T$, we indirectly want to give acceleration profiles to the system through the parameters K_p and K_d of the PD controller and we retrieve the input forces to achieve that. By extracting the acceleration part from the continuous dynamics function, we obtain:

$$\mathbf{a} = \begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix} = \begin{bmatrix} \ddot{x} \\ \ddot{y} + g \end{bmatrix} = \frac{1}{m} \begin{bmatrix} r_x f & -r_y \tau \\ r_y f & r_x \tau \end{bmatrix} = \frac{1}{m} \begin{bmatrix} r_x & -r_y \\ r_y & r_x \end{bmatrix} \begin{bmatrix} f \\ \tau \end{bmatrix}$$

$$\begin{bmatrix} f \\ \tau \end{bmatrix} = m \begin{bmatrix} r_x & r_y \\ -r_y & r_x \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{y} + g \end{bmatrix} = m \begin{bmatrix} r_x & r_y \\ -r_y & r_x \end{bmatrix} \begin{bmatrix} a_{x,des} \\ a_{y,des} + g \end{bmatrix}$$

We define the deviation from the desired position and velocity of the system as the errors: $\mathbf{e} = \mathbf{x}_{des} - \mathbf{x}$ and $\dot{\mathbf{e}} = \mathbf{v}_{des} - \mathbf{v}$ The desired acceleration profile through the PD parameters is: $\mathbf{a}_{des} = K_p\mathbf{e} + K_d\dot{\mathbf{e}}$ and the control inputs:

$$f = m(r_x a_{x,des} + r_y(a_{y,des} + g)) \tag{1}$$

$$\tau = m(-r_y a_{x,des} + r_x(a_{y,des} + g)) \tag{2}$$

As evidence from the equations (1),(2) above, by controlling the desired acceleration $\mathbf{a}_{des}$, we obtain the input forces to achieve that. We can test the correctness of the controller throughout an example while setting the desired pose of the system at $\mathbf{x}_{des} = \begin{bmatrix}1 & 3 & 0 & 0\end{bmatrix}^T$. The result of the simulation is displayed above for each of the integrators. The algorithms converge fast to the desired pose, while some integrators give slightly faster convergence than others.

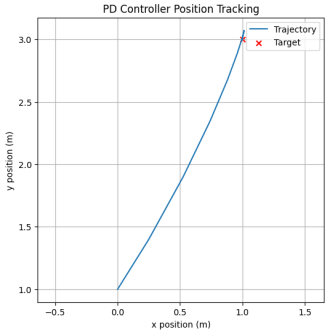


Fig. 1. Euler Integration

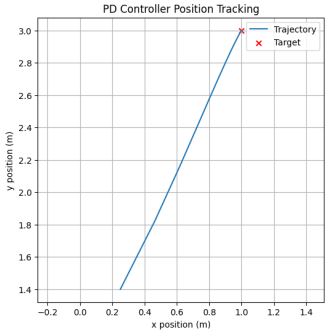


Fig. 2. Semi-Implicit Euler

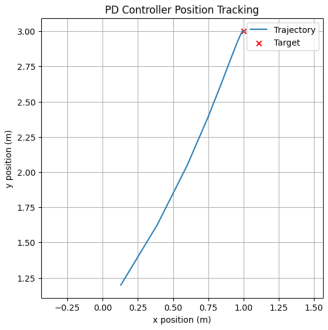


Fig. 3. RK4 Order Integration

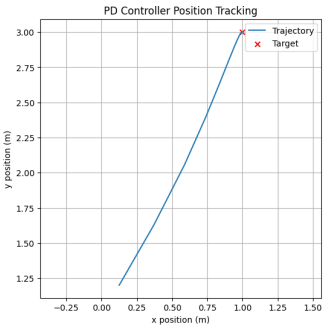


Fig. 4. Midpoint Integration

Comparing the four integrators, we notice that the Euler Integrations offer the slowest convergence. Specifically, with the Euler Integrator the desired state is achieved after 58 iterations and with the Semi-Implicit Euler Integrator after 59 iterations. The RK4 Order Integrator converges in 41 iterations while the Midpoint Integrator in only 40 iterations.

It should be pointed out that the simulation was set for $dt = 0.05sec$ and with PD parameters as $K_p = \begin{bmatrix}180 & 80\end{bmatrix}^T$ and $K_d = \begin{bmatrix}18 & 15\end{bmatrix}^T$ .The parameters are matrices consisting of relatively large numbers that can cause instability, but were chosen by trial and error for a given timestep dt. For better stability and robustness, a smaller dt should be used as well as smaller numbers on the K_p , K_d parameters.

III. Quadratic Programming via a Primal-Dual Augmented Lagrangian Method

A. Quadratic Programming

Before diving directly into the Primal-Dual ALM, a brief explanation of the QP problem should be given. Let us first remind of the general form of a Quadratic Programming problem.

$$\begin{aligned} \min_{\mathbf{x}} \ & f(\mathbf{x}) = \tfrac{1}{2}\mathbf{x}^T\mathbf{Q}\mathbf{x} + \mathbf{q}^T\mathbf{x} && \mathbf{x}, \mathbf{q} \in \mathbb{R}^N \text{ and } \mathbf{Q} > 0 \in \mathbb{R}^{NxN} \\ \text{s.t.} \ & \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0} && \mathbf{A} \in \mathbb{R}^{MxN} \text{ and } \mathbf{b} \in \mathbb{R}^M \\ & \mathbf{C}\mathbf{x} - \mathbf{d} \leq \mathbf{0} && \mathbf{C} \in \mathbb{R}^{PxN} \text{ and } \mathbf{d} \in \mathbb{R}^P \end{aligned}$$

We have N optimization variables, M equality constraints and P inequality constraints.
The $\mathbf{Q}$ is the quadratic term that penalizes large accelerations caused by input forces. The $\mathbf{x}^T\mathbf{Q}\mathbf{x}$ is the quadratic part that, if no constraints were involved, the minimum would be where the gradient is zero.

$$\nabla f(x) = \mathbf{Q}\mathbf{x} + \mathbf{q} = \mathbf{0} \Rightarrow \mathbf{x}^* = -\mathbf{Q}^{-1}\mathbf{q}$$

The computation of the term $\mathbf{Q}^{-1}$ is complex and arithmetically costly if the matrix $\mathbf{Q}$ is not convex. Nevertheless, the equation above indicates how fast the cost increases as we move away from the minimum. The $\mathbf{q}^T\mathbf{x}$ part is there to shift the minimum towards the desired state. It is a linear term that we can think of as a bias - it favors a desired target. The equality and inequality constraints act as limits to our system.

A full analysis of them will be given later on. Conceptually for our system, the QP problem answers to the question:

"What forces f and τ should we apply right now to move towards the desired pose with respect to the (physical) constraints of the system?"

B. Primal-Dual Augmented Lagrangian Method with Slack Variables

For our system, the QP problem is written as:

$$\min_{\mathbf{u}} f(\mathbf{u}) = \tfrac{1}{2}\mathbf{u}^T\mathbf{Q}\mathbf{u} + \mathbf{q}^T\mathbf{u}$$
$$\text{s.t. } \mathbf{A}\mathbf{u} - \mathbf{b} = \mathbf{0}$$

thus trying to find the minimum force $\mathbf{u} = \begin{bmatrix} f & \tau \end{bmatrix}^T$ for the hopper system to reach the desired target. The matrix $\mathbf{Q}$ penalizes large, energy-intensive forces while $\mathbf{q}$ derives from the previous input $\mathbf{u}_{prev}$ preventing sudden moves and ensuring smoothness of motion. This is the ideal model - no inequality constraints are involved and the equalities one derive from the nature of the problem through Newton's Second Law of motion.
In this case, the matrices $\mathbf{A}$ and $\mathbf{b}$ come from the equation:

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} + g \end{bmatrix} = \frac{1}{m}\begin{bmatrix} r_x & -r_y \\ r_y & r_x \end{bmatrix}\begin{bmatrix} f \\ \tau \end{bmatrix} \Rightarrow$$
$$\mathbf{A} = \frac{1}{m}\begin{bmatrix} r_x & -r_y \\ r_y & r_x \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} \ddot{x} \\ \ddot{y} + g \end{bmatrix}$$

The matrix $\mathbf{b}$ contains the desired acceleration profile for the system, which is given as before through the PD controller, while tuning the gains K_p and K_d accordingly. In reference to the matrices $\mathbf{C}$ and $\mathbf{d}$, they should be empty as there are no inequalities involved. If we are to introduce inequality constraints, these would arrive from physical limitations or realistic parameters for the input forces:

$$0 \le f \le f_{max} \quad -\tau_{max} \le \tau \le \tau_{max}$$
$$\begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}\begin{bmatrix} f \\ \tau \end{bmatrix} \le \begin{bmatrix} f_{max} \\ 0 \\ \tau_{max} \\ \tau_{max} \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{d} = \begin{bmatrix} f_{max} \\ 0 \\ \tau_{max} \\ \tau_{max} \end{bmatrix}$$

For the sake of the project, we set $C = [\,]$, $D = [\,]$ assuming no inequality constraints take in place. However, the algorithm is built to treat this kind of constraints and the analysis will be set on that factor.

Starting with the QP parameters, a slack variable s is introduced transforming all inequality constraints into equality ones since they are more easily interfaced with. The cost function now consists of the variables $\mathbf{x}$ and $\mathbf{s}$, called the Primal Variables. These are the "physical" variables - they can be directly adjusted to minimize the function f. The QP problem has changed to:

$$\min_{\mathbf{u},\mathbf{s}\ge 0} f(\mathbf{u}) = \tfrac{1}{2}\mathbf{u}^T\mathbf{Q}\mathbf{u} + \mathbf{q}^T\mathbf{u}$$
$$\text{s.t. } \mathbf{A}\mathbf{u} - \mathbf{b} = \mathbf{0}$$
$$\mathbf{C}\mathbf{u} + \mathbf{s} - \mathbf{d} = \mathbf{0}$$

We add two variables for each of the equality constraints $(\boldsymbol{\lambda}, \boldsymbol{\mu})$, known as the *Lagrange Multipliers*. These take the role of the Dual Variables and secure that no constraints are violated. If that happens, the multipliers increase trying to force back the solution into the feasible region. Intuitively, they act as a spring that, the more it is stretched, the harder it is to come back to its original state. A penalty term $\rho > 0$ is added to softly penalize constraint violations. By mixing all these, we get the Augmented Lagrangian function:

$$\mathcal{L}_\rho(\boldsymbol{x}, \boldsymbol{s}; \boldsymbol{\lambda}, \boldsymbol{\mu}) = \tfrac{1}{2}\boldsymbol{x}^T\boldsymbol{Q}\boldsymbol{x} + \boldsymbol{q}^T\boldsymbol{x} + \boldsymbol{\lambda}^T(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}) + \tfrac{\rho}{2}\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \boldsymbol{\mu}^T(\boldsymbol{C}\boldsymbol{x} + \boldsymbol{s} - \boldsymbol{d}) + \tfrac{\rho}{2}\|\boldsymbol{C}\boldsymbol{x} + \boldsymbol{s} - \boldsymbol{d}\|_2^2$$

$$QP : \tfrac{1}{2}\mathbf{u}^T\mathbf{Q}\mathbf{u} + \mathbf{q}^T\mathbf{u}$$
$$Lagr.Mult. : \boldsymbol{\lambda}^T(\mathbf{A}\mathbf{u} - \mathbf{b}) + \boldsymbol{\mu}^T(\mathbf{C}\mathbf{u} + \mathbf{s} - \mathbf{d})$$
$$Penalty : \tfrac{\rho}{2}\|\mathbf{A}\mathbf{u} - \mathbf{b}\|_2^2 + \tfrac{\rho}{2}\|\mathbf{C}\mathbf{u} + \mathbf{s} - \mathbf{d}\|_2^2$$

While keeping fixed the variables $(\boldsymbol{\lambda}, \boldsymbol{\mu})$, we compute the gradient of L_p with respect to $(\mathbf{u}, \mathbf{s})$. We get two first-order optimality conditions that we can solve over $\mathbf{u}$ and $\mathbf{s}$, thus updating the Primal variables. We can now update the Dual variables $(\boldsymbol{\lambda}, \boldsymbol{\mu})$ as:

$$\boldsymbol{\lambda}_{k+1} = \boldsymbol{\lambda}_k + \rho(\mathbf{A}\mathbf{u}_{k+1} - \mathbf{b})$$
$$\boldsymbol{\mu}_{k+1} = \boldsymbol{\mu}_k + \rho(\mathbf{C}\mathbf{u}_{k+1} + \mathbf{s}_{k+1} - \mathbf{d})$$

The core part of the ALM's algorithm has been applied. To check if the code approaches the saddle point $(\mathbf{u}^*, \mathbf{s}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ we introduce the KKT conditions. If they are satisfied, the algorithm has converged to the solution, and therefore the "searching" stops. In case the constraints stop improving, the penalty term ρ increases. This is the easiest way to penalize constraint violation for an ideal and simplified hopper robot system. The KKT conditions are as follows:

$$\|\mathbf{A}\mathbf{u}_{k+1} - \mathbf{b}\|_\infty < \epsilon_{feas}$$
$$\|\mathbf{C}\mathbf{u}_{k+1} + \mathbf{s}_{k+1} - \mathbf{d}\|_\infty < \epsilon_{feas}$$
$$\mathbf{Q}\mathbf{u}_{k+1} + \mathbf{q} + \mathbf{A}^T\boldsymbol{\lambda}_{k+1} + \mathbf{C}^T\boldsymbol{\mu}_{k+1} < \epsilon_{stat}$$

meaning that the algorithm stops when the constraints and the gradient of the cost function (w.r.t. $(\mathbf{u}, \mathbf{s})$) are satisfied by a precision number ϵ_{feas} and ϵ_{stat} respectively.

In conclusion, we can intuitively think of the system as this: The PD controller produces the desired acceleration profile and the matrix $\mathbf{b}$ is "reading" this. Depending on the inequality constraints, we fill the matrices $\mathbf{C}$ and $\mathbf{d}$ accordingly. Then, we find the optimal input force $\mathbf{u}_{opt}$ from the ALM solver that we use to update the vector $\mathbf{q} \rightarrow (\mathbf{u}_{prev} = \mathbf{u}_{opt})$. We finally integrate the system with respect to $\mathbf{u}_{opt}$ creating various simulations as seen above.

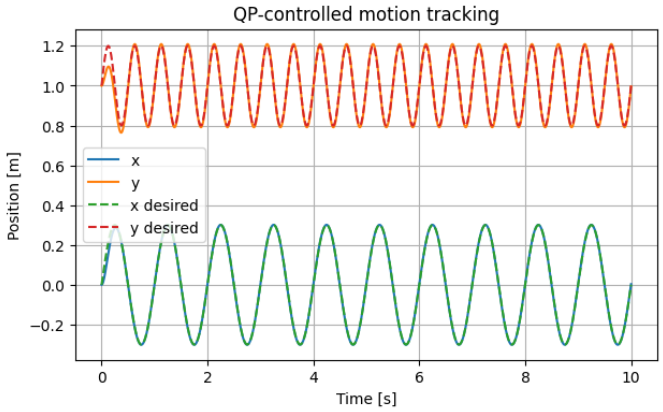


Fig. 5. $x = 0.3sin(\omega t),\ y = 1 + 0.2sin(2\omega t)$

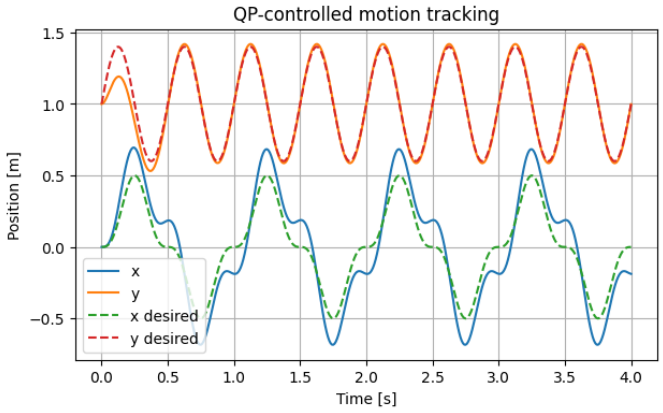


Fig. 6. $x = 0.5sin^3(\omega t),\ y = 1 + 0.4sin(2\omega t)$

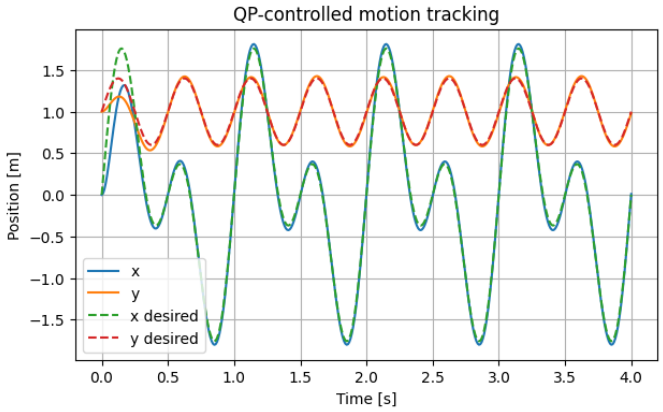


Fig. 7. $x = sin(\omega t) + sin(2\omega t),\ y = 1 + 0.4sin(2\omega t)$

Notes: The QP solver was implemented with the PD controller. This was on purpose so that the QP could find the right control input forces to achieve the acceleration profile given by the PD controller. We can remove it and put the PD behavior inside the cost function.

$$\min_{\mathbf{u}} f(\mathbf{u}) = \frac{1}{2}\mathbf{u}^T\mathbf{Q}\mathbf{u} + \mathbf{q}^T\mathbf{u} \quad with$$

$$\mathbf{Q} = \mathbf{A}^T\mathbf{A} \quad and \quad \mathbf{q} = -\mathbf{A}^T\mathbf{a}_{des}$$

For our case, the matrices were chosen as: $\mathbf{Q} = \mathbf{I}, \mathbf{q} = -\mathbf{Q}\mathbf{u}_{des}$ to transform the cost into:

$$\min_{\mathbf{u}} f(\mathbf{u}) = \frac{1}{2}\mathbf{u}^T\mathbf{Q}\mathbf{u} + \mathbf{q}^T\mathbf{u} = \min_{u} \frac{1}{2}(\mathbf{u} - \mathbf{u}_{des})^T\mathbf{Q}(\mathbf{u} - \mathbf{u}_{des})$$

From the simulations, we notice that, as the sinusidal's function difficulty increases, the QP solver finds it difficult to follow the trajectory.